Programmation Système
TP no 2 : Threads, synchonisations et atomicité

Guillaume Mercier
email : guillaume.mercier@bordeaux-inp.fr

2 Novembre 2026

Notions abordées dans ce TD :
- **gestion des threads :** `pthread_create`, `pthread_join`, `wait`,...
- **synchronisations inter-threads :**
  - verrous (`pthread_mutex_lock`, `pthread_mutex_unlock`);
  - variables de condition (`pthread_cond_wait`, `pthread_cond_signal`, `pthread_cond_broadcast`);
  - moniteurs;
  - atomicité des opérations et des variables.
- **mesures et analyse des temps d'exécution**
- **gestion et attribution des ressources de calcul d'une machine (*binding***

Les codes sont disponibles sur mon compte : `/net/npers/mercier/Cours/IF210/TP/TP2`.

1 Verrous : la théorie . . . et la pratique

Question 1
Compilez le programme `exo1/test_mutex_types.c` et exécutez-le : que constatez-vous ? Allez faire un tour sur la page de manuel des fonctions `pthread_mutex_init`, `pthread_mutex_lock/unlock` et `pthread_mutexattr_init` pour confirmer ou infirmer vos hypothèses.

Question 2
Comparez le comportement d'un *fast* mutex avec celui d'un sémaphore possédant un unique jeton.

2 Partage de variable : synchronisation, optimisation, atomicité, etc.

2.1 Synchronisation de threads avec une variable

1. Examinez le code du programme `exo2/first.c` puis compilez-le sans optimiser le code (i.e. sans l'option `-O1`, `-O2` ou `-O3` et enfin exécutez-le. Le programme fonctionne-t-il comme attendu ?

2. Recompilez ce même programme mais avec l'option de compilation `-O2` et exécutez cette nouvelle version.
   — Que constatez-vous ?
   — Quelle explication pouvez-vous donner du phénomène ?
   — Quelle(s) solution(s) pouvez-vous proposer ? Implémentez-les et vérifiez qu'elles sont effectivement fonctionnelles.

2.2 Variable partagée entre threads

2.2.1 Exposition du problème

1. Écrivez un programme dans lequel deux threads incrémentent chacun une variable globale partagée un certain nombre de fois (vous commencerez par 1000). Par exemple quelque chose comme :

```
#define NTIMES (1000)

int x = 0; //variable globale

void *thread_func(void *arg)
{
  for(int i  = 0; i < NTIMES; i++)
    x++;
  pthread_exit(NULL);
}
```

   Le thread principal (`main`) doit afficher la valeur de cette variable globale une fois que les deux threads ont terminé leur exécution.
2. Compilez et exécutez ce programme.
   — La valeur de la variable est-elle correcte ?
   — Quelle conclusion pouvez-vous en tirer ?
3. Afin de confirmer cette conclusion, augmentez significativement le nombre d'itérations (par exemple 1000000) et réexécutez le programme.
   — La valeur de la variable globale est-elle toujours cohérente ?
   — Comment pouvez-vous expliquer cette situation ?
4. Essayer de corriger le problème en appliquant les solutions suivantes :
   — Qualifier la variable globale en `volatile`. Est-ce que cela fonctionne ?
   — Compiler le code en l'optimisant (par exemple `-O2`). Est-ce que cela corrige le problème ? Pourquoi ? (indice : vous pourrez examiner le code assembleur généré par le compilateur avec une commande de type `objdump`).

2.2.2 Correction(s) du problème

Il est cependant anormal que l'exécution correcte d'un programme dépende de son niveau d'optimisation de compilation. En effet, l'optimisation ne devrait déboucher que sur un programme plus rapide, pas sur un programme *correct*. Cette solution n'en est donc pas vraiment une.

Nous allons maintenant examiner des solutions acceptables permettant de régler le problème et ce quel que soit le nombre d'itérations ou le niveau d'optimisation de compilation considérée.
En fait, le problème est formulable très simplement : l'opération de post-incrémentation (ou pré-incrémentation) ++ n'est pas *atomique* car il s'agit d'une lecture suivie d'une écriture (à la différence d'une affection qui ne consiste qu'en une écriture). Quand le premier thread commence cette opération (la partie lecture), le second thread peut lui aussi modifier la variable *avant* que le premier ne finisse la partie écriture. On se retrouve donc avec une variable dans un état incohérent. Il s'agit donc bien d'une **section critique** qui doit être protégée. Deux solutions s'offrent à vous :

1. Utiliser un verrou pour protéger l'accès à la variable
2. Utiliser une variable atomique

1. Protection avec un verrou

Modifiez votre programme pour que l'accès à la variable soit protégée par un verrou (de type `pthread_mutex_t`). Vérifiez que l'exécution est maintenant correcte, en faisant varier le nombre d'itérations ainsi que le niveu d'optimisation de compilation du code.

2. Utilisation d'une variable atomique

Enlevez le verrou et modifiez votre programme pour que la variable soit maintenant atomique :

1. une première version est de passer le type de la variable globalae en `sig_atomic_t`. Est-ce que cela fonctionne ? Que pouvez-vous en déduire ?
2. une seconde version est de qualifier la variable d'atomique avec le mot-clef `_Atomic` (ne pas oublier d'inclure `<stdatomic.h>` et de compiler avec `-latomic`). Est-ce que cela fonctionne ? Que pouvez-vous en déduire ?

Bonus. Reprenez l'exercice de la section 2.1 en qualifiant la variable d'atomique. Est-ce que cela fonctionne ? Pourquoi ?

3. Comparaison des performances

Comparez les temps d'exécution des deux solutions fonctionnelles à l'aide la commande `time`.

3 Réentrance (*Thread Safety*) vs. *Async-signal-safety*

(Re)lisez les sections 7.3.2 et 8.2.4 du support de cours et réflechissez aux raisons qui poussent le manuel :

1. à classer certaines fonctions comme n'étant *pas async-signal-safe*
2. à indiquer que seules des fonctions *async-signal-safe* doivent être utilisées après un `fork` dans un processus multithreadé.

4 Mesure des temps d'exécution d'un programme multithreadé

La commande `time` affiche trois mesures de temps :
— `real` le temps total d'exécution. Il s'agit du *wallclock time*, la différence entre le temps de fin de programme et le temps de début d'exécution du programme.

— `user` le temps CPU pris par le programme en mode actif *utilisateur*
— `sys` le temps CPU pris par le programme en mode actif *noyau*

Mais quelles peuvent être les relations entre ces trois mesures de temps ? Dans tous les cas, on considèrera que la machine qure vous utilisez est complètement libre, i.e qu'il n'y a pas de processus d'autres utilisateurs qui y occupent des ressources (vérifiez que c'est bien le cas).

4.1 Première hypothèse à tester : real $\simeq$ (user + system)

1. Compilez et mesurez les temps d'exécution du programme `exo3/prog_exemple.c` (qui crée un fichier, y écrit un certain nombre de caractères puis détruit ce fichier [1]) à l'aide de la commande `time`.
   — l'hypothèse est-elle confirmée ?
   — exécutez plusieurs fois ce programme pour en être bien sûr(e)s.
2. Vous allez maintenant forcer l'exécution du programme sur un processeur en particulier, à l'aide de la commande `hwloc-bind` (syntaxe `hwloc-bind -p pu:`*numéro de processeur*). Pour connaître le nombre de processeurs/cœurs disponible sur votre machine, vous utiliserez la commande `lstopo-no-graphics`. Avant de relancer le programme `prog_exemple`, vous allez lancer le programme `occupe_cpu` **sur le même processeur que va utiliser `prog_exemple`**. Là encore vous utiliserez la commande `hwloc-bind` pour forcer l'exécution du programme `occupe_cpu`.
   — l'hypothèse est-elle de nouveau confirmée ?
   — comment expliquez-vous ce comportement ?

4.2 Seconde hypothèse à tester : real $\geq$ (user + system)

1. Écrivez un programme qui crée un nombre variable de threads (ce nombre sera un argument du programme). Tous ces threads vont exécuter la même fonction dont voici le code :

```
void *thread_func(void *arg)
{
  int toto = 0;
  while(++toto);
  pthread_exit(NULL);
}
```

2. Mesurez le temps d'exécution de ce programme en faisant varier le nombre de threads de 1 à *nprocs* où *nprocs* est le nombre de processeurs/cœurs de la machine sur laquelle vous travaillez.
   — Que constatez-vous ?
   — Pouvez-vous en déduire une formule faisant intervenir *real*, *user*, *system* ainsi que le nombre de threads créés ?
3. Par défaut, un programme multithreadé utilise tous les cœurs disponibles. Forcez l'exécution du programme afin qu'il n'occupe qu'une partie des ressources avec la commande `hwloc-bind`. Par exemple, pour n'occuper que les quatre premiers cœurs de votre

1. Le but de ce programme est accessoire.

machine, la commande de *binding* sera `hwloc-bind -p pu:0-3 ./mon_programme`. Faites varier ce nombre de cœurs (1, 2, la moitié des cœurs disponibles, etc.) et mesurez à chaque fois les temps d'exécution de votre programme.

— Que remarquez-vous au niveau du temps *user*, d'une exécution à l'autre ?
— Que remarquez-vous au niveau du temps *real*, d'une exécution à l'autre ?
— Pouvez-vous en déduire une formule faisant intervenir *real*, *user*, *system* ainsi que le nombre de threads créés *et* le nombre de ressources utilisées ? On supposera que les temps *user* t_u et *system* t_s sont identiques pour tous les threads et ce quelle que soit l'exécution.

5 Parallélisation avec OpenMP

OpenMP est standard de parallélisation basé sur le multithreading (qui utilise en interne les pthreads). OpenMP repose à la fois sur des *directives de compilation* (`#pragma omp`) et sur une bibliothèque contenant des fonctions. Pour créer un programme OpenMP, il faut donc inclure le fichier d'en-tête `<omp.h>` et utiliser l'option de compilation `-fopenmp`. Si cette option n'est pas utilisée, les directives de compilation d'OpenMP seront tout simplement ignorées (et le programme obtenu sera donc séquentiel et non parallèle).
La documentation de la dernière mouture du standard est disponible ici : (`https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf`).
Dans un premier temps, vous allez vous familiariser (un peu) avec OpenMP et son fonctionnement.

1. examinez le code du programme `construct_parallel.c`, compilez-le et enfin exécutez-le en faisant varier le nombre de threads (un argument du programme). Que se passe-t-il quand on ne passe aucun argument au programme ?
2. examinez maintenant le code du programme `construct_for.c` et essayez d'en comprendre le fonctionnement (compilez-le et exécutez-le, bien entendu).

Maintenant, vous allez paralléliser le programme `exo4_omp/calcul_min_array.c` qui calcule le minimum d'un tableau de *m* entiers. Vous pourrez vous inspirer de de la solution de l'exercice numéro 68 (`exo4_omp/calcul_min_array_mt.c`) qui utilise explicitement des pthreads. Pour se faire, vous vous aiderez de la documentation du standard OpenMP et notamment :
— la construction `parallel` : p. 384
— la construction `for` : p. 416
— la construction `critical` : p. 299

Vérifiez que les résultats obtenus entre la version séquentielle, la version pthreads et la version OpenMP sont cohérents.